



MOBILNE APLIKACIJE

Vežbe 5

Servisi i prijemnici poruka

2025/2026

Sadržaj

1. Servisi	3
1.1 Životni ciklus servisa	3
1.2 Pravljenje background servisa	5
1.2 Pravljenje foreground servisa	6
2. Notifikacije	8
2.1 Kreiranje notifikacija	8
2.2 Kreiranje notifikacija	9
3. Asinhroni zadaci	10
3.1 Pravljenje asinhronog zadatka	10
3.2 Handler i Looper	11
4. Prijemnici poruka	13
4.1 Pravljenje prijemnika poruka	13
5. Zakazivanje zadataka	16
6. Domaći	18

1. Servisi

Tokom programiranja, često ćete imati priliku da se susretnete sa pojmom *servis*. Šta on tačno podrazumeva u kontekstu Androida? Pod servisom podrazumevamo operacije koje ne zahtevaju interakciju korisnika, već se izvršavaju u pozadini. Koristi se za izvršavanje akcija kao što su: puštanje muzike, rad sa učitavanjem i ispisom fajlova, preuzivanje fajlova, itd.

Postoje 3 vrste servisa:

- *Background servis*
- *Foreground servis*
- *Bound servis*

Background servis se pokreće samo kada je aplikacija pokrenuta, tako da će biti prekinut kada se aplikacija prekine. Izvršava se neodređeno vreme u pozadini i zaustavlja se kada završi operaciju. Android operativni sistem može da ga ubije ukoliko mu ponestane RAM/baterije. Neophodno je da bude eksplicitno kreiran od strane aktivnosti ili fragmenta koji ga poziva. Ne pruža korisnicima nikakvu informaciju o tome kakvu operaciju izvršava.

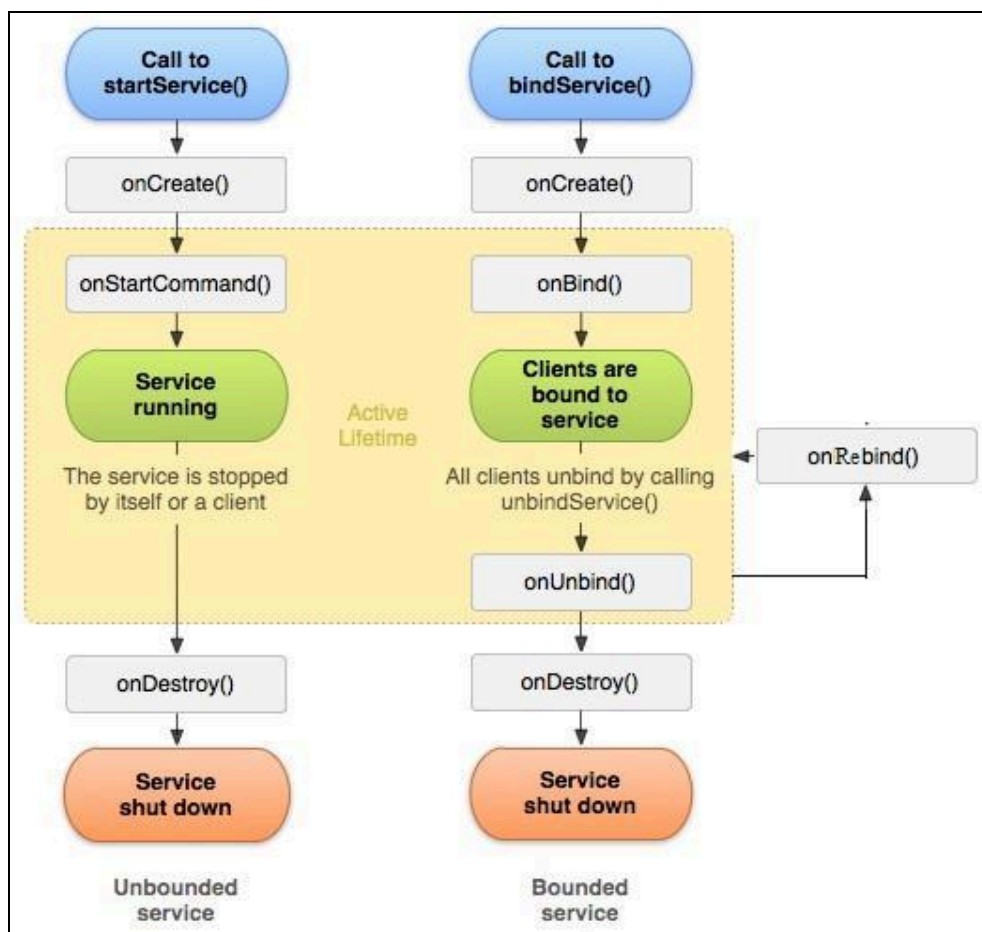
Za razliku od *Background servisa*, *Foreground servis* je servis koji ostaje živ čak i kada je aplikacija prekinuta. Korisnicima pruža informaciju o izvršavanoj operaciji putem notifikacija. Notifikacije treba da se prikazuju i kada aplikacija nije više u fokusu.

Bound servis ili vezan servis, se izvršava samo dok je komponenta za koju je vezan i dalje aktivna. On nudi interfejs koji omogućava komponentama da komuniciraju sa njim (šalju se zahtevi i dobijaju se odgovori).

1.1 Životni ciklus servisa

Isto kao što aktivnosti imaju svoj životni ciklus, tako i servisi imaju svoj. Životni ciklus servisa je jednostavniji od životnog ciklusa aktivnosti. On poseduje različite metode koje se pozivaju prilikom prelaska iz jednog u drugo stanje. Na slici 1 možemo da vidimo životni ciklus servisa.

Početak životnog veka je od metode *onCreate*, a završetak je sa metodom *onDestroy*. Servis je aktivan (aktivni životni vek) od metode *onStartCommand* ili *onBind*, pa sve do metode *onDestroy* ili *onUnbind*.



Slika 1. Životni ciklus servisa

OnCreate

Ova metoda se poziva nakon što neka komponenta zatraži pravljenje servisa. Ovde navodimo samu svrhu servisa.

onStartCommand

Kada neka komponenta želi da kreira servis, ona pozove metodu *startService*. U tom trenutku sistem će pozvati metodu *onStartCommand*.

onBind

Kada neka komponenta zatraži da se napravi vezani servis ona poziva metodu *bindService*. Nakon metode *bindService* poziva se metoda *onBind*.

onUnbind

Metoda *onUnbind* se poziva posle poziva *unbindService* metode.

onRebind

onRebind pozivamo posle poziva *bindService*, ako je prethodno izvršena *onUnbind* metoda.

onDestroy

Na samom kraju, pozivamo metodu *onDestroy* prilikom uništavanja servisa.

Servis može da se zaustavi pozivom metode *stopSelf*, može neka druga komponenta da ga zaustavi pozivom metode *stopService* ili ga zaustavlja Android da bi oslobodio memoriju.

1.2 Pravljenje *background* servisa

Da bismo što bolje razumeli šta je servis, napravićemo ga. Pravimo novu klasu sa nazivom *SyncService* koja nasleđuje klasu *Service*. Po uzoru na aktivnosti, servis registrujemo u *AndroidManifest* datoteci (slika 2).

```
<service
    android:name=".services.SyncService"
    android:exported="false" />
```

Slika 2. Navođenje servisa u *AndroidManifest*-u

Servis poseduje metodu *onStartCommand* koja se poziva prilikom izvršavanja zadatka servisa (slika 3). U našem primeru servis će, ako su zadovoljeni uslovi, pokrenuti asinhroni zadatak.

U klasi *CheckConnectionTools* kreirali smo pomoćnu metodu, koja poziva metode Android operativnog sistema i proverava da li je uređaj povezan na internet, i ako jeste na koji je to način povezan. Ta pomoćna metoda kao rezultat vraća jedan od brojeva 0, 1 ili 2, u zavisnosti od toga da li je uređaj povezan na wifi, da li koristi mobilne podatke ili nije povezan na internet. Ovu pomoćnu metodu pozivamo u servisu i povratnu vrednost čuvamo u *connectionStatus*-u (linija 31). U slučaju da je uslov zadovoljen, tj. da je uređaj povezan na internet, pokrećemo asinhroni zadatak (linija 34).

```
27     @Override
28     public int onStartCommand(Intent intent, int flags, int startId) {
29         Log.i(TAG, msg: "Service started");
30
31         int connectionStatus = CheckConnectionTools.getConnectivityStatus(getApplicationContext());
32
33         if (isConnected(connectionStatus)) {
34             executor.execute(this::performBackgroundWork);
35         } else {
36             Log.i(TAG, msg: "No internet connection");
37         }
38
39         return START_NOT_STICKY;
40     }
```

Slika 3. Metoda servisa *onStartCommand*

1.2 Pravljenje *foreground* servisa

Za kreiranje *foreground* servisa, deklariramo komponentu u *AndroidManifest.xml* i navodimo tip željenog servisa. U našem slučaju, kreiramo servis koji omogućava reprodukciju zvuka (slika 4).

```
<service
    android:name=".services.ForegroundService"
    android:foregroundServiceType="mediaPlayback"
    android:enabled="true"
    android:exported="false" />
```

Slika 4. Deklarisanje *foreground* servisa za reprodukciju zvuka

Servis se pokreće/zaustavlja klikom na *switch* komponentu unutar *SettingsFragment*-a, prilikom čega se kreiraju namere u kojima navodimo naziv servisa. Namere, zatim, prosleđujemo *startForegroundService()* i *startService()* metodama sa načenim kodom za pokretanje i zaustavljanje servisa (*ACTION_START* i *ACTION_STOP*) (slika 5).

```
99     private void setupSwitch() {
100         startServiceSwitch.setOnCheckedChangeListener((CompoundButton buttonView, boolean isChecked) -> {
101
102             Context ctx = requireContext();
103             Intent intent = new Intent(ctx, ForegroundService.class);
104             if (isChecked) {
105                 intent.setAction(ForegroundService.ACTION_START);
106                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
107                     ContextCompat.startForegroundService(ctx, intent);
108                 } else {
109                     ctx.startService(intent);
110                 }
111             } else {
112                 intent.setAction(ForegroundService.ACTION_STOP);
113                 ctx.startService(intent);
114             }
115         });
116     }
117 }
```

Slika 5. Pokretanje i zaustavljanje servisa

Sam servis treba da pokrene plejer za reprodukciju zvuka i da omogući pauziranje i zaustavljanje reprodukcije. Ovo nam omogućava *startForegroundInService()* metoda unutar *ForegroundService* klase (slika 6.). Osim pokretanja plejera, metoda se brine i o definisanju notifikacija koje su neophodne za rad sa *Foreground* servisima o kojima će biti više reči u narednom odeljku.

```

92     private void startServiceInForeground() {
93         Log.i(TAG, msg: "Starting foreground service");
94         initPlayer();
95         startForeground(NOTIFICATION_ID, buildNotification());
96     }
97
98     1 usage
99     private void initPlayer() {
100         if (player == null) {
101             player = MediaPlayer.create(context: this, Settings.System.DEFAULT_RINGTONE_URI);
102             player.setLooping(true);
103         }
104         player.start();
105     }

```

Slika 6. Pokretanje plejera

Unutar *onStartCommand()* metode, u zavisnosti od prosleđene akcije unutar namere, okidamo različite operacije (slika 7).

ACTION_START poziva gore pomenutu metodu i pokreće reprodukovanje zvuka. Korisniku se prikazuje notifikacija sa mogućnostima klika na *PLAY*, *PAUSE* i *STOP*.

ACTION_PLAY omogućava ponovno pokretanje zvuka nakon pauziranja ili stopiranja (klikom na *PAUSE* ili *STOP* preko notifikacije).

ACTION_PAUSE pauzira zvuk, dok *ACTION_STOP* stopira reprodukovanje i zaustavlja *foreground* servis nakon čega se uklanja i notifikacija.

```

38 public int onStartCommand(Intent intent, int flags, int startId) {
39     String action = intent != null ? intent.getAction() : null;
40     if (action == null) return START_NOT_STICKY;
41     Log.i(TAG, msg: "Action: " + action);
42
43     switch (action) {
44         case ACTION_START:
45             startServiceInForeground();
46             break;
47
48         case ACTION_PLAY:
49             play();
50             break;
51
52         case ACTION_PAUSE:
53             pause();
54             break;
55
56         case ACTION_STOP:
57             stop();
58             break;
59     }
60
61     return START_NOT_STICKY;
62 }

```

Slika 7. Pokretanje različitih operacija unutar servisa

2. Notifikacije

Notifikacije su poruke koje Android prikazuje korisniku mimo korisničkog interfejsa same aplikacije. Služe za obaveštavanje korisnika o događajima unutar samih aplikacija, prikaz podsetnika, itd. Obično se prikazuju u gornjem delu ekrana uređaja gde je moguće proširiti, ukloniti, reagovati ili izvršiti određenu akciju nad njima, kao i pokrenuti aplikaciju od koje je stiglo obaveštenje.

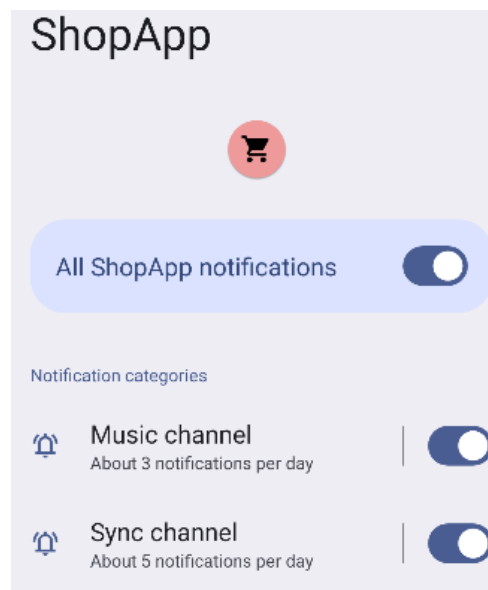
2.1 Kreiranje notifikacija

Da bi se notifikacije mogle prikazivati, neophodno je kreirati kanal putem kojih će pristizati. Kanal za slanje i prijem notifikacija sadrži ime, opis i definisan prioritet notifikacija u zavisnosti od kod se određuje način prikazivanja samih poruka. Kreiranje kanala prikazano je na slici 8.

```
170     private void createNotificationChannel(String name, String description, String id, int importance) {  
171         if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return;  
172  
173         NotificationChannel channel = new NotificationChannel(id, name, importance);  
174         channel.setDescription(description);  
175  
176         NotificationManager manager = getSystemService(NotificationManager.class);  
177         if (manager != null) {  
178             manager.createNotificationChannel(channel);  
179         }  
180     }
```

Slika 8. Kreiranje notifikacionog kanala

U aplikaciji je moguće imati više kanala što korisniku omogućava kontrolu nad vizuelnim i audio podešavanjima notifikacija koje pripadaju jednoj grupi. Unutar naše aplikacije kreirana su dva notifikaciona kanala: MUSIC_CHANNEL_ID i SYNC_CHANNEL_ID. Korisnik može omogućiti i onemogućiti prikaz svih notifikacija koje su vezane za ove kanale posebno (slika 9) kroz sistemska podešavanja.



Slika 9. Podešavanja prikaza notifikacija

2.2 Kreiranje notifikacija

Kao što je već napomenuto, unutar *startForegroundService()* metode, definiše se notifikacija koja će se prikazati korisniku nakon pokretanja plejera (slika 10).

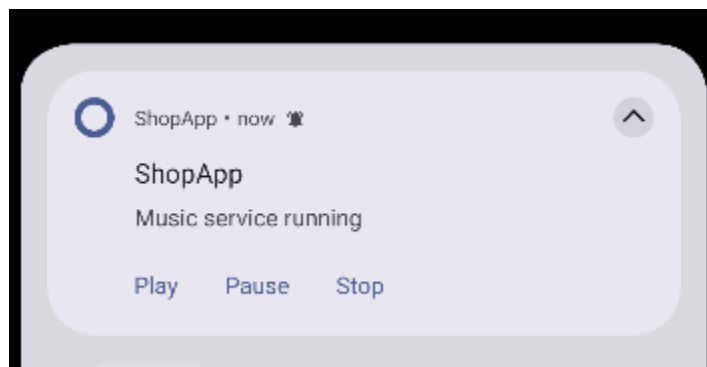
Za sam prikaz notifikacije neophodno je kreirati osnovni izgled notifikacije i preuzeti kanal putem kog će notifikacija biti prikazana (linije 105-112). Notifikaciji se može dodeliti font, veličina slova, proslediti tekst, podesiti ikonica i veličina ikonice, kao i prioritet.

Notifikacijama se mogu podesiti i akcije putem kojih korisnik može da odreaguje na pristiglo obaveštenje. Linije 116-120 prikazuju definisanje *PLAY* akcije. Kreira se namera kojoj se postavlja akcija *ACTION_PLAY*. Unutar prethodno pomenute *onStartCommand()* metode *foreground* servisa (slika 7) očekuje se pristizanje ovakve akcije i reaguje se pokretanjem plejera.

```
105  @ private NotificationCompat.Builder baseNotificationBuilder() {
106      return new NotificationCompat.Builder( context: this, HomeActivity.MUSIC_CHANNEL_ID)
107          .setContentTitle("ShopApp")
108          .setContentText("Music service running")
109          .setSmallIcon(R.mipmap.ic_launcher)
110          .setOngoing(true)
111          .setOnlyAlertOnce(true);
112  }
113
114  1 usage
115  @ private Notification buildNotification() {
116      PendingIntent playPi = PendingIntent.getService(
117          context: this, requestCode: 1,
118          new Intent( packageContext: this, ForegroundService.class).setAction(ACTION_PLAY),
119          PendingIntent.FLAG_IMMUTABLE
120      );
121
122      PendingIntent pausePi = PendingIntent.getService(
123          context: this, requestCode: 2,
124          new Intent( packageContext: this, ForegroundService.class).setAction(ACTION_PAUSE),
125          PendingIntent.FLAG_IMMUTABLE
126      );
127
128      PendingIntent stopPi = PendingIntent.getService(
129          context: this, requestCode: 3,
130          new Intent( packageContext: this, ForegroundService.class).setAction(ACTION_STOP),
131          PendingIntent.FLAG_IMMUTABLE
132      );
133
134      return baseNotificationBuilder()
135          .addAction(android.R.drawable.ic_media_play, title: "Play", playPi)
136          .addAction(android.R.drawable.ic_media_pause, title: "Pause", pausePi)
137          .addAction(android.R.drawable.ic_menu_close_clear_cancel, title: "Stop", stopPi)
138          .build();
139  }
```

Slika 10. Kreiranje notifikacije

Na slici 11 prikazana je ovako kreirana notifikacija.



Slika 11. Notifikacija sa definisanim akcijama

3. Asinhroni zadaci

Android poseduje glavnu nit (*MainThread*) koja crta korisnički interfejs i odgovara na interakciju korisnika. Da ne bismo došli u situaciju da blokiramo glavnu nit i time blokiramo UI, radnje koje se sporo izvršavaju smeštamo na drugu nit.

Glavna nit je jedina koja može da ažurira UI, te je potrebno da se sa podacima, iz radnji koje su bile duge, vratimo na nju.

Asinhrono zadatke koristimo da bismo olakšali asinhrono izvršavanje operacija. Asinhroni zadaci automatski izvršavaju blokirajuću operaciju u sporednoj, pozadinskoj niti i vraćaju rezultat UI niti. Svi asinhroni zadaci jedne aplikacije izvršavaju se u jednoj niti.

3.1 Pravljenje asinhronog zadatka

Kod starijih verzija API level-a (< 30) koristila se klasa *AsyncTask* za kreiranje asinhronih zadataka. Nakon nasleđivanja klase *AsyncTask*, mogli smo da redefinišemo i primenimo metode *onPreExecute*, *doInBackground*, *onProgressUpdate* i *onPostExecute*.

Od verzije API level (>= 30) ova klasa se više ne koristi i postoji mogućnost kreiranja asinhronog zadatka pomoću *ExecutorService* klase.

Potrebno je prvo napraviti instancu *ExecutorService*-a i kreirati nit (*thread*) pomoću metode *newSingleThreadExecutor()*. Primer dat na slici 12.

```
private final ExecutorService executor = Executors.newSingleThreadExecutor();
```

Slika 12. Metoda *newSingleThreadExecutor()*

Pomoću instance *executor* moguće je izvršiti asinhron zadatak metodom *execute()* u okviru koje se interno implementira *Runnable* funkcionalni interfejs koji je namenjen za pokretanje niti i definiše metod *run()* koji se poziva bez argumenata u okviru pozadinske niti (background), gde

se izvršava sav posao koji dugo traje. Novije verzije androida ne traže ručno definisanje ove metode (slika 13).

```
int connectionStatus = CheckConnectionTools.getConnectivityStatus(getApplicationContext());

if (isConnected(connectionStatus)) {
    executor.execute(this::performBackgroundWork);
} else {
    Log.i(TAG, msg: "No internet connection");
}
```

Slika 13. Pokretanje zadatka u drugoj niti

Kako bismo pokrenuli određene procese na glavnoj UI niti, moguće je pomoću *Handler*-a pozvati metodu *post()* u okviru koje se, takođe, interno pokreće metoda *run()* koja sada pokreće zadatak u okviru glavne UI niti (slika 14).

```
47     private void performBackgroundWork() {
48         Log.i(TAG, msg: "Background work started");
49
50         try {
51             Thread.sleep( millis: 1000);
52         } catch (InterruptedException e) {
53             Thread.currentThread().interrupt();
54             Log.e(TAG, msg: "Thread interrupted", e);
55         }
56
57         mainHandler.post(this::sendResult);
58     }
```

Slika 14. Metoda *runOnUiThread()*

Da biste izvršili operaciju na UI threadu iz servisa, možete koristiti Handler vezan za glavni looper (*Looper.getMainLooper()*). To omogućava slanje poruke ili pokretanje *Runnable* zadatka na UI threadu, što je ključno za izmene korisničkog interfejsa ili druge operacije koje moraju biti izvršene na UI threadu.

3.2 Handler i Looper

Ako želimo da ponovo koristimo nit (na primer: da izbegnemo stvaranje novih niti i smanjimo svoj memorijski prostor), moramo da je zadržimo u životu i da nekako osluškuje nova uputstva. Uobičajeni način da se to postigne jeste kreiranje petlje (*Looper*), u ovom primeru, unutar niti *run()* metode. *Looper* održava svoju nit živom, u ovom primeru glavnu nit. Niti podrazumevano nemaju vezan *Looper* za sebe, ali ga možemo kreirati. Može postojati samo jedan *Looper* po niti.

Da bismo mogli da koristimo *Looper* i pokrenemo zadatke unutar UI niti potreban nam je *Handler*. *Handler* je odgovoran za dodavanje poruka u red *Looper*-a, a kada dođe njihovo vreme, odgovoran je za izvršavanje istih poruka na *Looper*-ovoj niti. Kada se kreira *Handler* on je usmeren ka određenom *Looper*-u tj. ka određenoj niti. Metoda *post()* kreira poruku i dodaje je na kraj reda *Looper*-a. Možemo kreirati *Handler* koji je vezan za određeni *Looper* na sledeći način dat na slici 15.

Ovaj pristup omogućava komunikaciju između pozadinskog servisa i UI thread-a bez rizika od izazivanja izuzetaka zbog nepravilnog pristupa korisničkom interfejsu iz pozadinskog thread-a.

```
private final Handler mainHandler = new Handler(Looper.getMainLooper());
```

Slika 15. Kreiranje *handler*-a

4. Prijemnici poruka

Prijemnik poruka (*BroadcastReceiver*) je komponenta koja obrađuje i odgovara na poruke, koje dobije od drugih aplikacija ili samog sistema. Sistem šalje poruke kada je npr. baterija prazna, ekran isključen itd. Aplikacije emituju obaveštenja kada su npr. neki podaci uspešno skinuti sa interneta.

Prijemnici poruka se često koriste u sprezi sa servisima i asinhronim zadacima i najčešće samo obaveštavaju druge komponente da počnu sa izvršavanjem određenih zadataka.

Prijemnici poruka mogu da obrađuju 2 vrste događaja:

- *Normalni događaji* – asinhroni su; prijemnici ih obrađuju nedefinisanim redosledom; efikasniji su.
- *Uređeni događaji* – obrađuju se redom; svaki prijemnik može da prosledi događaj sledećem prijemniku ili da potpuno obustavi njegovu obradu.

onReceive:

Prijemnik poruka postoji samo u toku izvršavanja metode *onReceive*, te se u njoj ne mogu izvršavati asinhronne operacije (prikazivanje dijaloga, vezivanje za servis..).

Ova metoda se poziva iz glavne niti, te duge operacije treba izvršavati u posebnoj servisu, koji startuje posebnu nit.

4.1 Pravljenje prijemnika poruka

Pravimo novu klasu *SyncReceiver* koja nasleđuje *BroadcastReceiver* i u metodi *onReceive* kreiramo ponašanje. Isto kao i servise, prijemnike poruka navodimo u *AndroidManifest* datoteci (slika 16). Za svaki *Receiver* možemo definisati koju poruku će osluškivati pomoću `<intent-filter>` taga.

```
<receiver
    android:name=".receivers.SyncReceiver"
    android:exported="false">
    <intent-filter>
        <action android:name="com.example.shopapp.SYNC_DATA"/>
    </intent-filter>
</receiver>
```

Slika 16. Navođenje prijemnika poruka u *AndroidManifest*-u

Pre nego što pogledamo šta će tačno raditi naš prijemnik poruka, nameće se pitanje kako ćemo uopšte da mu šaljemo poruke?

U našem asinhronom zadatku, koji smo napravili u prethodnom poglavlju, u okviru metode *handler.post()* napravili smo *Intent* i definisali akciju „SYNC DATA“ (slika 17). Akciju smo definisali zato što jedan prijemnik poruka može da prima više poruka iz aplikacije. Uz tu poruku smo

definisali i `RESULT_CODE` parametar koji prosleđujemo intent-u sa statusom konekcije. Na samom kraju, pozivamo metodu `sendBroadcast` i prosleđujemo joj kreiranu nameru (*intent*).

```
60     private void sendResult() {
61         Log.i(TAG, "msg: \"Sending result to UI\"");
62
63         int latestStatus = CheckConnectionTools.getConnectivityStatus(getApplicationContext());
64
65         Intent intent = new Intent(packageContext: this, SyncReceiver.class);
66         intent.setAction(HomeActivity.SYNC_DATA);
67         intent.putExtra(RESULT_CODE, latestStatus);
68         sendBroadcast(intent);
69
70         stopSelf();
71     }
```

Slika 17. Metoda asinhronog zadatka koja šalje poruku prijemniku poruka

Ako se vratimo na našu klasu *SyncReceiver*, ona u metodi *onReceive* prima nameru (*intent*), koja će čuvati akciju i podatke, koji su stigli (slika 18). Na prethodnoj slici smo videli da asinhroni zadatak prosleđuje prijemniku poruka nameru, a sada vidimo i gde ta namera stiže.

Naš prijemnik poruka će reagovati samo ako je dobio “SYNC DATA”, te u if-u proveravamo da li je prijemniku upravo takva poruka stigla.

```
30     public void onReceive(Context context, Intent intent) {
31
32         if (intent == null || !HomeActivity.SYNC_DATA.equals(intent.getAction())) {
33             return;
34         }
```

Slika 18. Metoda *onReceive*

U slučaju da nam je stigla poruka „SYNC DATA“, preuzimamo sadržaj namere. Dalje ponašanje prijemnika poruke zavisi od sadržaja koji je stigao, tj. informacije da li je uređaj povezan na internet i ako jeste na koji način (slika 19). U našem slučaju, korisniku prištiže notifikacija sa informacijom o konekciji.

```

38     int resultCode = intent.getIntExtra(SyncService.RESULT_CODE, defaultValue: -1);
39
40     NotificationManagerCompat notificationManager =
41         NotificationManagerCompat.from(context);
42
43     PendingIntent wifiSettingsIntent = PendingIntent.getActivity(
44         context,
45         NOTIFICATION_ID,
46         new Intent(Settings.ACTION_WIFI_SETTINGS),
47         PendingIntent.FLAG_IMMUTABLE
48     );
49
50     NotificationCompat.Builder builder =
51         new NotificationCompat.Builder(context, HomeActivity.SYNC_CHANNEL_ID);
52
53     Bitmap largeIcon;
54
55     switch (resultCode) {
56
57         case CheckConnectionTools.TYPE_NOT_CONNECTED:
58             largeIcon = BitmapFactory.decodeResource(
59                 context.getResources(),
60                 R.drawable.ic_action_network_wifi
61             );
62
63             builder.setSmallIcon(R.drawable.ic_action_about)
64                 .setContentTitle("Automatic Sync problem")
65                 .setContentText("Bad news, no internet connection")
66                 .setContentIntent(wifiSettingsIntent)
67                 .addAction(
68                     R.drawable.ic_action_network_wifi,
69                     "Turn wifi on",
70                     wifiSettingsIntent
71                 );
72             break;

```

Slika 19. Ponašanje prijemnika poruke u zavisnosti od sadržaja koji mu je stigao

Bitno je još napomenuti da smo u metodi *onCreate* klase *MainActivity* pozvali napravljenu metodu *setUpReceiver* (slika 20). Ta metoda inicijalizuje prijemnik poruka i definiše nameru, koju želimo da izvršavamo kada dođe za to vreme.

```

182     private void setUpReceiver() {
183         syncReceiver = new SyncReceiver();
184         Log.i(Log.TAG_RECEIVER, SYNC_DATA);
185         IntentFilter filter = new IntentFilter(SYNC_DATA);
186
187         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
188             registerReceiver(syncReceiver, filter, RECEIVER_NOT_EXPORTED);
189         } else {
190             registerReceiver(syncReceiver, filter);
191         }
192     }

```

Slika 20. Metoda *setUpReceiver*

5. Zakazivanje zadataka

Timer je komponenta koja služi za zakazivanje jednokratnih zadataka ili zadataka koji se ponavljaju. Svaki timer ima jednu nit koja zadatke izvršava sekvencijalno. Kada aplikacija nije aktivna, timer neće izvršiti svoj posao.

Za demonstraciju *timer*-a (slika 21) koristimo uvodni ekran za korisnika (*splash screen*). Timer zakazujemo tako što pozivamo metodu *schedule* i prosleđujemo joj *TimerTask*, sa metodom *run*, i vreme koliko dugo *timer* treba da čeka da bi se posao izvršio. U metodi *run* smo napravili eksplicitnu nameru, prelazak sa aktivnosti *SplashScreenActivity* na aktivnost *MainActivity*.

```
15      @Override
16      protected void onCreate(Bundle savedInstanceState) {
17          super.onCreate(savedInstanceState);
18          setContentView(R.layout.activity_splash);
19          /*...*/
22          int SPLASH_TIME_OUT = 1000;
23          new Timer().schedule(() -> {
24              /*...*/
34              Intent intent = new Intent( packageContext: SplashScreenActivity.this, HomeActivity.class);
35              /*...*/
39              startActivity(intent);
40              /*...*/
44              finish();
45          }, SPLASH_TIME_OUT);
47      }
```

Slika 21. Primer upotrebe *timer*-a

Klasa *AlarmManager* nam omogućuje da pristupimo sistemskom alarmu i da pokrenemo aplikaciju u nekom trenutku u budućnosti.

U metodi *scheduleSync()* koristimo *AlarmManager* i definišemo kada je potrebno da se ponavlja (slika 22).

Svaki put kada dođe trenutak da se pokrene zadatak, emitovaće se objekat klase *Intent* koji će reći OS-u šta treba da se uradi.

Definišemo na koliko vremena treba neki zadatak da se uradi, a u narednim linijama kako da manager reaguje. Manager je definisan tako da je u režimu ponavljanja, da kreće odmah da meri vreme i da reaguje na svaki minut.

Svi zadaci koji se ponavljaju nisu uvek egzaktni, što znači da se najverovatnije neće izvršiti tačno na svakih n minuta, već na $n + x$ minuta, gde je x neko malo odstupanje.

```

211     private void scheduleSync() {
212         if (alarmManager == null) return;
213
214         Intent intent = new Intent(packageContext: this, SyncService.class);
215
216         PendingIntent pendingIntent = PendingIntent.getService(
217             context: this,
218             requestCode: 0,
219             intent,
220             flags: PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
221         );
222
223         long triggerTime = System.currentTimeMillis();
224
225         alarmManager.setRepeating(
226             AlarmManager.RTC_WAKEUP,
227             triggerTime,
228             SYNC_INTERVAL,
229             pendingIntent
230         );
231
232         Log.i(tag: "HOME ACTIVITY", msg: "Sync scheduled");
233         showToast(text: "Alarm Set");
234     }

```

Slika 22. Primer upotrebe *AlarmManager* klase

6. Domaći

Domaći se nalazi na *Canvas-u* (*canvas.ftn.uns.ac.rs*) na putanji *Вежбе/05 Задачамак.pdf*.

Primer možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije/>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić (milan.jovkic@uns.ac.rs)
- Jelena Matković (matkovic.jelena@uns.ac.rs)